

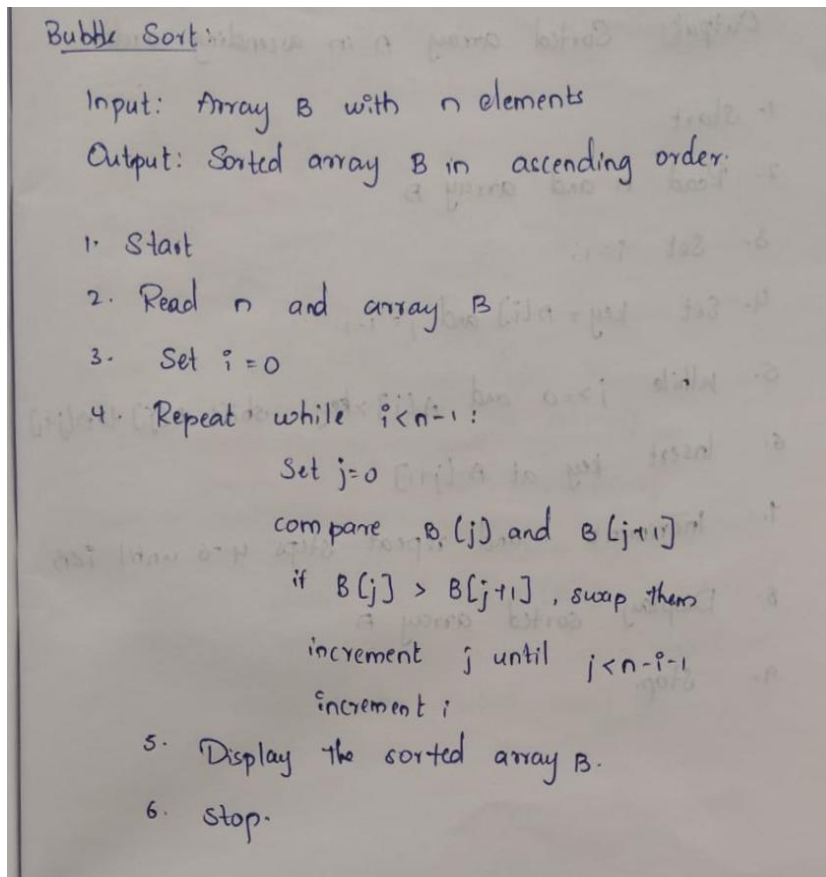


PRACTICAL: 01

Aim: To implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

BUBBLE SORT

Algorithm:



Program / Code:

```
#include <iostream>
#include <vector> using
namespace std;

void bubbleSort(vector<int> &arr)
{
```

```
int n = arr.size();
for(int i=0;i<n-1;i++)
for(int j=0;j<n-i-1;j++)
    if(arr[j]>arr[j+1])
        swap(arr[j],arr[j+1]);
}

int main() {
    vector<int> numbers = {64, 34, 25, 12, 22, 11, 90};
    bubbleSort(numbers);

    for (int value : numbers)
        cout << value << ' ';

    cout << '\n';
    return 0;
}
```

Output:

```
PS C:\Users\pusap\Downloads> C:\Users\pusap\Downloads\c++\bubble_sort.exe
11 12 22 25 34 64 90
PS C:\Users\pusap\Downloads>
```

Time Complexity:



Marwadi University
Marwadi Chandarana Group



Faculty of Engineering and Technology
Department – CSE AI / ML 01AI0506 - Design
and Analysis of Algorithms

Bubble Sort:

for n elements $\rightarrow n-1$ comparisons.

1st $\rightarrow n-1$
2nd $\rightarrow n-2$
:
:
:
 $n-1 \rightarrow 1$

for Best Case:
The loop will iterate only $(n-1)$ time
 $T.C \rightarrow O(n)$

$\Rightarrow (n-1)(n-2)(n-3) \dots 1 = \frac{n(n-1)}{2}$

for Average case = worst case
for bubble sort $= \frac{n^2 - n}{2}$

$\Rightarrow$ Worst case. $T(n) = O(n^2)$

SELECTION SORT

Algorithm:

Selection Sort

Input: Array A with n elements
Output: Sorted array A in ascending order.

1. Start.
2. Read n and array A .
3. Set $i=0$.
4. Repeat while $i < n-1$:
 - Set $min = i, j = j+1$
 - compare $A[j]$ with $A[min]$
 - if $A[j] < A[min]$ set $min = j$
 - increment j until $j < n$
 - swap $A[i]$ and $A[min]$
 - increment i
5. Display the sorted array A .
6. Stop.

Program / Code:

```
#include <iostream>
#include <vector> using
namespace std;

void selectionSort(vector<int> &arr)
{   int n=arr.size();

    for(int i=0;i<n-1;i++)
    {
        intmin=i;

        for(int j=i+1;j<n;j++)
        if(arr[j]<arr[min])
            min=j;

        swap(arr[i],arr[min]);
    }
}

int main() {
    vector<int> numbers = {64, 34, 25, 12, 22, 11, 90};
    selectionSort(numbers);
    cout<<"Sorted array using selection sort: \n";
    for (int value : numbers)
        cout << value << ' ';

    cout << "\n";
    return 0;
}
```

Output:

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\selectionsort.exe
Sorted array using selection sort:
11 12 22 25 34 64 90
PS C:\Users\pusap\Downloads\c++> █
```

Time Complexity:

Selection Sort:

```
for (int i=0; i<n-1; i++) → n-1
{
    int minIndex = i; → 1
    for (int j=i+1; j<n; j++) → n(n-1)/2
    {
        if (arr[j] < arr[minIndex]) → 1
        {
            minIndex = j;
        }
    }
    swap(arr[i], arr[minIndex]);
}
```

outer loop: The outer loop runs from $i=0$ to $n-1$
So it executes $n-1$ times.

inner loop: pass comparisons

pass	comparisons
1	$n-1$
2	$n-2$
3	$n-3$
$\vdots$	$\vdots$
$n-1$	1

best case: $T(n) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$

$T(n) = n-1 + 1 + \frac{n^2-n}{2} + 1 + 1 + 1 + 1$
 $= n + \frac{n^2-n}{2} + 4 = \frac{2n + n^2 - n + 8}{2} = \frac{n^2 + n + 8}{2}$
 $T(n) = O(n^2)$

Average case:

for random array

```
for (i=0; i<n-1; i++) → n-1
{
    int min=i
    for (j=i+1; j<n; j++) → n(n-1)/2
    {
        if (arr[j] < arr[min])
        {
            min=j;
        }
    }
    swap(arr[i], arr[min]);
}
```

$T(n) = \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2-n}{2}$
 $T(n) = O(n^2)$



Best case: If the array is sorted it checks every element.
 $T(n) = O(n^2)$

INSERTION SORT

Algorithm:

Insertion Sort
Input: Array A with n elements.
Output: Sorted array A in ascending order.

1. Start
2. Read n and array A.
3. Set $i = 1$.
4. Set $key = A[i]$ and $j = i - 1$.
5. While $j \geq 0$ and $A[j] > key$, shift $A[j]$ to $A[j+1]$.
6. Insert key at $A[j+1]$.
7. Increment i and repeat steps 4-6 until $i < n$.
8. Display sorted array A.
9. Stop.

Program / Code:

```
#include <iostream>
#include <vector> using
namespace std;
```

```
void insertionSort(vector<int> &arr)
{
    int n=arr.size();

    for(int i=1;i<n;i++)
    {
        int key=arr[i];
        int j=i-1;

        while(j>=0 && arr[j]>key)
        {
            arr[j+1]=arr[j];
            j--;
        }

        arr[j+1]=key;
    }
};

int main(){
    vector<int> numbers = {64, 34, 25, 12, 22, 11, 90};
    insertionSort(numbers);

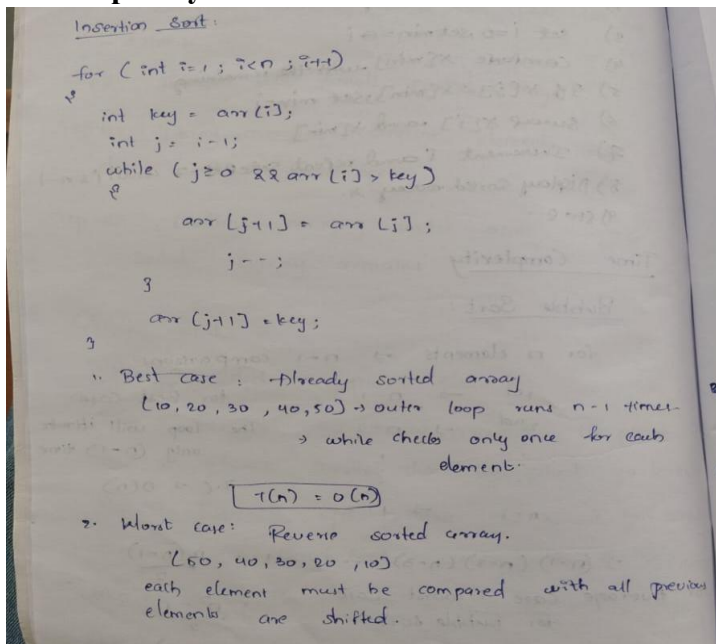
    cout << "Sorted array using insertion sort: \n";
    for (int value : numbers)
        cout << value << ' ';

    cout << "\n";
    return 0;
}
```

Output:

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\insertion_sort.exe
Sorted array using insertion sort:
11 12 22 25 34 64 90
PS C:\Users\pusap\Downloads\c++>
```

Time Complexity:



QUICK SORT Algorithm:

Quick SORT :

- Input: Array A with n elements
Output: Sorted array A in ascending order
1. Start
 2. Read n and array A
 3. Select the last element as the pivot.
 4. Set $i = \text{low} - 1$
 5. Compare each element with the pivot.
 6. If $A[i] < \text{pivot}$, increment i and swap $A[i]$ and $A[j]$
 7. Swap $A[i+1]$ with the pivot.
 8. Apply Quick sort to the left and right subarrays.
 9. Display the sorted array A .
 10. Stop.

Program / Code:

```
#include <iostream>
#include <vector> #include
<algorithm> using namespace
std;
```

```
int partition(vector<int> &arr, int low, int high)
{   int pivot = arr[high];
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
    {   if (arr[j] < pivot)
        {   i++;
            swap(arr[i], arr[j]);
        }
    }
```

```
    swap(arr[i + 1], arr[high]);
```

```
return i + 1;
}

void quickSort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};
    int n = static_cast<int>(arr.size());

    quickSort(arr, 0, n - 1);

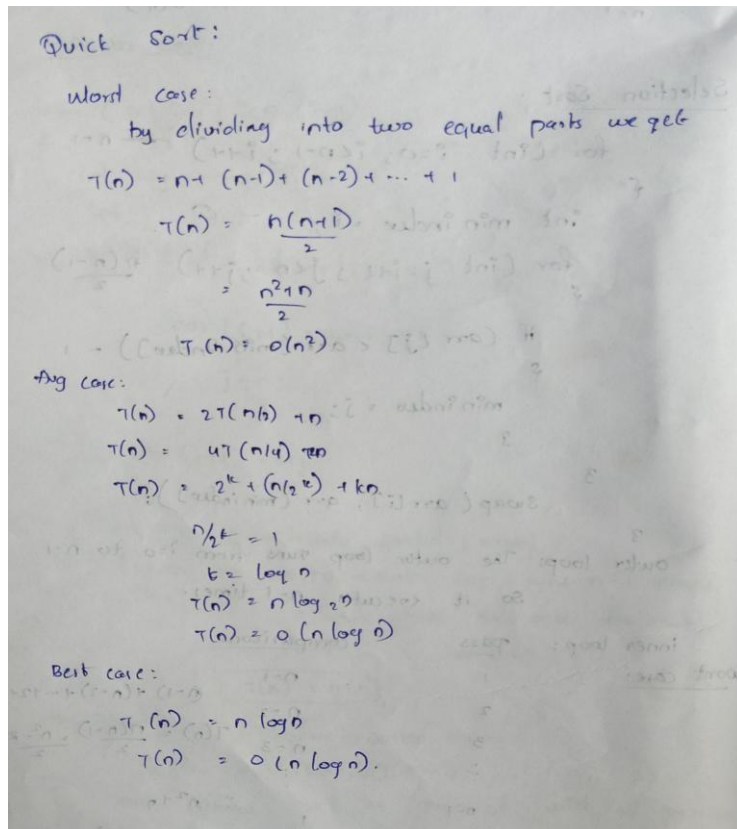
    cout << "Sorted array using quick sort:\n";
    for (int value : arr)
        cout << value << ' ';

    cout << '\n';
    return 0;
}
```

Output:

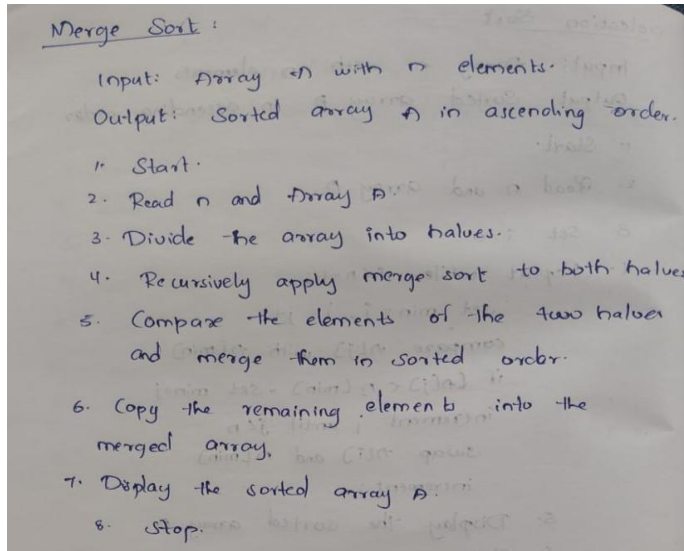
```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\quick_sort.exe
Sorted array using quick sort:
11 12 22 25 34 64 90
PS C:\Users\pusap\Downloads\c++>
```

Time Complexity:



MERGE SORT

Algorithm:



Program/code:

```
#include <iostream> #include  
<vector> using namespace  
std;
```

```
void merge(vector<int> &arr,int l,int m,int r)  
{ int n1=m+1;  
int n2=r;
```

```
vector<int> L(n1),R(n2);
```

```
for(int i=0;i<n1;i++)  
L[i]=arr[l+i];
```

```
for(int j=0;j<n2;j++)  
R[j]=arr[m+1+j];
```

```
int i=0,j=0,k=1;
```

```
while(i<n1 && j<n2)
```

```
        {      if(L[i]<=R[j])
arr[k++]=L[i++];
else
arr[k++]=R[j++];
    }

    while(i<n1)
arr[k++]=L[i++];

    while(j<n2)
arr[k++]=R[j++];
}

void mergeSort(vector<int> &arr,int l,int r)
{   if(l<r)
    {       int m=(l+r)/2;

        mergeSort(arr,l,m);
        mergeSort(arr,m+1,r);

        merge(arr,l,m,r);
    }
}

int main() {
    vector<int> numbers = {64, 34, 25, 12, 22, 11, 90};
    mergeSort(numbers, 0, numbers.size()
- 1);

    cout << "Sorted array using merge sort: \n";
    for (int value : numbers)
    cout << value << ' ';
```



```
cout << '\n';  
return 0;  
}
```

Output:

```
PS C:\Users\pusap\Downloads\c++> & C:\Users\pusap\Downloads\c++\merge_sort.exe  
Sorted array using merge sort:  
11 12 22 25 34 64 90  
PS C:\Users\pusap\Downloads\c++>
```

Time Complexity:

Merge sort:

$$T(n) = 2T(n/2) + n$$
$$T(n) = [2T(n/4) + n/2] + n$$
$$T(n) = 4T(n/8) + 3n$$
$$T(n) = 8T(n/16) + 7n$$
$$T(n) = 2^k T(n/2^k) + kn$$
$$\frac{n}{2^k} = 1$$
$$2^k = n$$
$$k = \log_2 n$$
$$T(n) = nT(1) + n \log_2 n$$
$$T(n) = n + n \log_2 n$$
$$T(n) = O(n \log n)$$

Best case $O(n \log n)$

Avg case $O(n \log n)$

worst case $O(n \log n)$

Conclusion:

Merge Sort and Quick Sort are efficient divide-and-conquer sorting algorithms. Merge Sort provides a consistent $O(n \log n)$ time complexity and is stable, while Quick Sort is generally faster in practice with an average $O(n \log n)$ complexity but can take $O(n^2)$ in the worst case. Both algorithms are useful for efficiently sorting large datasets.